



Full Length Article

Enhanced trajectory reconstruction from sparse and noisy GPS data: A progressive chunked transformer approach

Yonghui Liu^a, Qian Li^b, Inhi Kim^{a,*}^a Cho Chun Shik Graduate School of Mobility, Korea Advanced Institute of Science and Technology, Daejeon, 34051, Republic of Korea^b Donghai Laboratory, Zhoushan, 316021, China

ARTICLE INFO

Keywords:

Trajectory reconstruction
Transformer
Chunked processing
Heuristic-informed
Parallel computing

ABSTRACT

Trajectory reconstruction from sparse and noisy GPS data is critical for applications such as urban mobility analysis, transportation planning, and navigation systems. However, large sampling intervals and the typically long output sequences required to reconstruct coherent travel trajectories significantly increase computational complexity, particularly in the presence of noise. To address these challenges, we propose a progressive chunked transformer (ProChunkFormer), which is a deep learning method for trajectory reconstruction that employs self-attention mechanisms and chunked processing to balance efficiency with accuracy. ProChunkFormer first generates intermediate trajectories at a semi-high frequency from low-frequency sampled data, and then the remaining trajectory is divided into manageable blocks and reconstructed parallelly in the condition of the semi-high-frequency trajectory. By combining progressive reconstruction with chunk processing, ProChunkFormer not only mitigates the cumulative errors commonly observed in autoregressive models but also alleviates the rapid increase in complexity associated with reconstructing ultralong trajectories. Specifically, our approach achieves quadratic optimization in time and space for attention modules, with cubic time savings compared with autoregressive decoding. A case study using an open-source taxi trajectory dataset confirms the effectiveness of our approach. The performance of ProChunkFormer is comparable to that of autoregressive transformers while offering better running efficiency. It improves the accuracy, F1 score (F1), mean absolute error (MAE), and road network mean absolute error (MAE_RN) by 23.1%, 18.6%, 22.3%, and 25.1%, respectively, for trajectories with a long interval time of up to 240 s. Furthermore, we investigate incorporating heuristic information to guide trajectory reconstruction for each block. The experimental results indicate an improvement in both the overall performance and convergence speed of the model.

1. Introduction

With the rapid development of surveillance devices such as global positioning systems (GPSs), radar, and cameras, increasing amounts of data are being gathered from vehicles and roads, which makes it possible to track the trajectories of road users to implement various transportation applications such as traffic prediction (Li et al., 2021; Meng et al., 2022; Sun et al., 2019; Xu et al., 2023; Zheng et al., 2020) and travel time estimation (Abbar et al., 2020; Mazloumi et al., 2010; San-ullah et al., 2016; Sang et al., 2024; Woodard et al., 2017).

However, as noted in previous works, two issues exist with the collected trajectory. First, trajectory data often include noise for various reasons, such as satellite signal errors, environmental factors, and human errors. This can significantly affect the accuracy and reliability of

the data. For example, in the case of GPS trajectory data, some points may be situated beyond the confines of the road. Second, many organizations (e.g., taxi and shared bicycle operators) do not publish complete trajectory data because of privacy protocols or efforts to reduce technical resources (Zhao and Zhang, 2019). The trajectory data usually only report the status at the start time and end time of a trip (Chondrogiannis et al., 2022). Additionally, there is an issue of low sampling, which results in the continuously sampled positions being far apart from each other (Chen et al., 2023; Yuan et al., 2010). Consequently, it is difficult for most existing methods developed for the applications mentioned above to utilize these trajectories effectively owing to noise (Fig. 1a) and uncertainty (Fig. 1b–d) problems.

To enhance trajectory data to support urban applications more effectively, researchers have proposed numerous trajectory

* Corresponding author.

E-mail address: inhi.kim@kaist.ac.kr (I. Kim).<https://doi.org/10.1016/j.commtr.2025.100200>

Received 27 December 2024; Received in revised form 1 April 2025; Accepted 9 April 2025

Available online 2 August 2025

2772-4247/© 2025 The Authors. Published by Elsevier Ltd on behalf of Tsinghua University Press. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

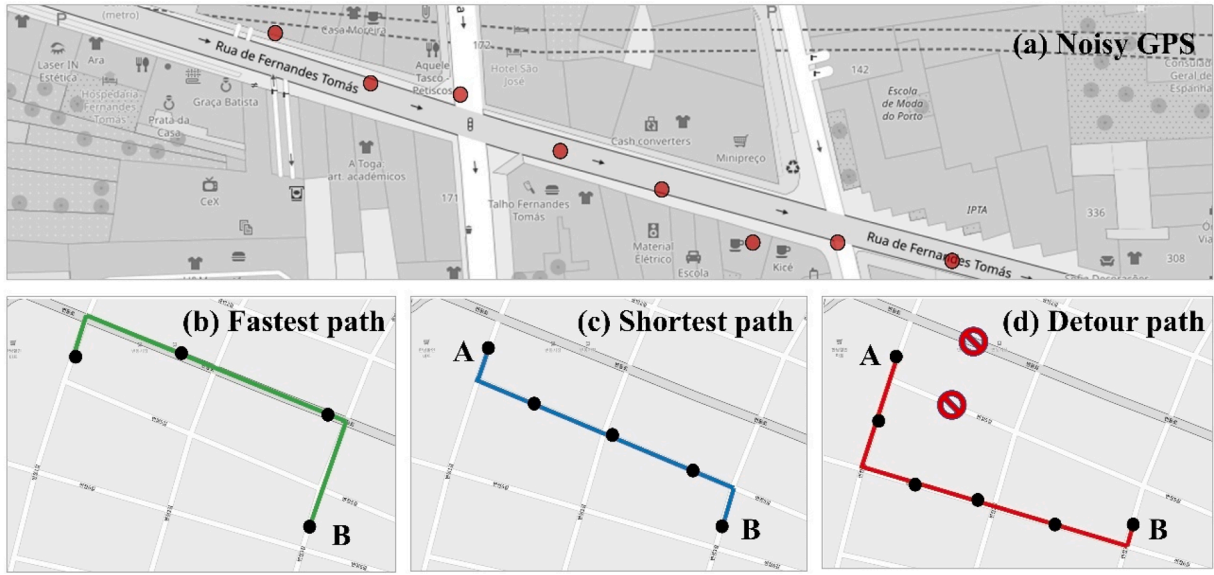


Fig. 1. Noise and sparseness of the GPS. (a) Noise of the GPS. (b–d) Trajectory uncertainty between two points, A and B, due to the possibility of taking different paths. Map data © OpenStreetMap contributors, licensed under CC BY-SA 2.0.

reconstruction methods that aim to reconstruct true trajectories from noisy and incomplete data (Rafati Fard et al., 2017; Rao et al., 2018; Sun and Ban, 2013; Zhao et al., 2024). Initial studies on this problem always divided the process into two stages. Points are first interpolated between the sparsely sampled trajectory to reduce uncertainties. Then, a hidden Markov model (HMM) map-matching technique (Newson and Krumm, 2009) is employed to map the completed trajectories onto the road. Various models have been proposed for incomplete trajectory interpolation, employing heuristic rule-based methods and deep learning technologies (Feng et al., 2018; Hoteit et al., 2014; Li et al., 2019; Xia et al., 2021). Rule-based methods typically rely on empirical observations and simplifications of road users' movements, such as the assumption of uniform speeds and the preference for the shortest path. However, owing to these simplifications, they are unable to consider additional aspects necessary for comprehending intricate trajectory patterns and dynamics. In contrast, deep learning models, which often employ the Seq2Seq architecture for solving this problem, are trained on large-scale data and can integrate various data sources, including environmental features and historical data such as weather conditions, day of the week, and historical trajectories. As a result, these models generally achieve superior performance compared with heuristic rule-based models. These two-stage solutions can yield favorable outcomes and provide a certain degree of interpretability. However, there may be concerns over the accumulation of errors. Recently, a multitask learning model has been proposed to address this problem in an end-to-end manner. The model predicts the road segment and the moving ratio on this road segment simultaneously for each time step, resulting in a map-matched trajectory directly.

Overall, despite extensive research in the field, recovering trajectories from low-sampling noise data poses significant challenges.

- 1) **Expansive search space.** In the case of recovering a trajectory with a long time interval, multiple potential paths exist between two recorded points, complicating the task further. Previous studies have evaluated the performance of various trajectory reconstruction methods across different sampling intervals. The findings reveal a distinct trend: These methods perform well with small sampling intervals but often struggle to accurately match sparse trajectory data with larger temporal gaps.
- 2) **High computational complexity.** When attempting to recover trajectories over long time periods, i.e., when the target sequence length

is large, the HMM algorithm becomes time-consuming. Similarly, the autoregressive decoder architecture in deep learning Seq2Seq models causes inefficiency in such circumstances.

- 3) **Error Accumulation.** For two-stage methods, errors in missing point interpolation can cause inaccuracies in map matching. Sequential or autoregressive models (e.g., standard Seq2Seq) generate a long sequence of predicted points by conditioning on previously predicted points. Over many iterations, small inaccuracies can compound into severe trajectory errors.

To address the abovementioned problems, this study proposes a progressive chunked transformer (ProChunkFormer) for trajectory reconstruction. Unlike conventional two-stage methods that handle missing point interpolation and map matching separately, our framework employs a multitask learning strategy to directly predict map-matched trajectories. By integrating these processes into an end-to-end architecture, ProChunkFormer effectively eliminates intermediate error propagation, ensuring a more robust and accurate reconstruction. A key innovation of ProChunkFormer is its chunked processing strategy, which divides long, noisy trajectories into smaller, manageable segments. This segmentation alleviates the quadratic complexity growth inherent in traditional transformer-based attention mechanisms, thereby enhancing computational efficiency when processing lengthy sequences. Moreover, because each segment can be processed independently, the design naturally supports parallel computation, significantly increasing both the efficiency and scalability. By localizing the error accumulation within individual segments and preventing local inaccuracies from propagating throughout the entire sequence, this approach also improves the reconstruction accuracy for sparse trajectory data.

Additionally, our framework incorporates heuristic prior knowledge to further improve the reconstruction accuracy. Specifically, it leverages the shortest path between consecutive recorded points as auxiliary information, utilizing road network topology to refine trajectory estimation. This integration of deep learning with structured heuristics helps reduce the uncertainty introduced by sparse sampling, ensuring a more reliable and context-aware reconstruction. By balancing efficiency, accuracy, and interpretability, ProChunkFormer offers a scalable and practical solution for low-density GPS trajectory reconstruction in real-world applications.

The contributions of this study are summarized as follows.

- 1) End-to-end trajectory reconstruction under map constraints. This study adopts a multitask learning architecture for end-to-end trajectory reconstruction, effectively leveraging road network information. A transformer-based module is employed to capture the spatiotemporal characteristics of both the road network and the trajectories.
- 2) Chunk processing for sparse, long-duration data. ProChunkFormer addresses the challenges associated with sparse and long-duration trajectories. The case study demonstrates that this method enhances both the spatial and temporal efficiency of trajectory reconstruction, with only a slight negative impact on overall performance. Notably, in cases of extremely sparse sampled trajectories, the performance improves, as chunked processing effectively reduces the error accumulation typically observed in autoregressive decoders.
- 3) Integration of heuristic prior knowledge. To better address the uncertainty in sparse trajectories, we incorporate heuristic information, such as the shortest path between two consecutive points, as prior knowledge in the progressive reconstruction process. The experimental results demonstrate its effectiveness in improving trajectory reconstruction.

This paper is organized as follows: Section 2 reviews the literature related to trajectory reconstruction and Seq2Seq models. Section 3 details the proposed model. Section 4 introduces a case study in Porto to demonstrate the effectiveness of the proposed model. Section 5 presents discussions and provides limitations and potential research directions. Section 6 concludes this study.

2. Literature review

2.1. Overview of trajectory reconstruction via GPS data

Trajectory reconstruction aims to reconstruct the position of a moving object over time on the basis of partial or noisy observations, which is a common problem in real-world scenarios (Shi et al., 2021; Wang et al., 2024; Xu et al., 2023). From this recovered information, we can precisely estimate the spatial and temporal characteristics of moving road users and transportation systems, which is important in various applications, including autonomous driving and traffic analysis.

Despite the widespread use of vehicle GPS data in trajectory reconstruction, multiple challenges arise from measurement noise, which stems from technological limitations or environmental interference, and uncertainty at different scales, including spatial (path-level and lane-level) and temporal (speed and acceleration) variations (Li et al., 2017; Lin et al., 2025a; Liu et al., 2020; Sanaullah et al., 2016). While dense sensor networks allow for near-continuous tracking, low-penetration conditions, where only a small fraction of vehicles transmit trajectory data or report infrequently, result in extended sampling intervals and pose significant challenges for accurate trajectory reconstruction. Addressing these challenges requires effective methodologies capable of handling noise, sparse data, and computational efficiency constraints.

Recovering precise trajectories from incomplete and noisy GPS data involves two key tasks. First, missing positional information needs to be completed between consecutive sparse GPS points, where multiple options may exist, and the most likely trajectory should be selected as the final output. Second, the GPS data must be matched to the road network. This is essential because GPS data often contain errors, causing the indicated position to potentially deviate from the actual road.

2.2. Traditional methods

Early methods for trajectory reconstruction often rely on explicit modeling assumptions, heuristics, or probabilistic frameworks to handle missing data and noise. Given an incomplete noisy trajectory, one straightforward method involves mapping the points to the nearest road

segments (Hoteit et al., 2014) and then completing the missing information, assuming that road users are moving at a uniform speed between two consecutive points along the shortest path. However, this method often fails in real-world applications in areas with densely packed road networks (Newson and Krumm, 2009), as the nearest map matching method, which considers individual points, is likely to be mapped to incorrect road segments. To overcome this limitation, some studies have considered sequences of points before finalizing a match (Greenfeld, 2002; White et al., 2000), with hidden Markov models (HMMs) playing a significant role in resolving map-matching issues (Krumm et al., 2007; Newson and Krumm, 2009). For example, Newson and Krumm (2009) introduced a novel algorithm based on an HMM that maps points to road networks and simultaneously selects the most likely root. Nevertheless, the effectiveness of such methods diminishes with lower sampling rates.

Building on probabilistic foundations, advancements in filtering techniques such as Kalman filters and particle filters have markedly refined the process of trajectory reconstruction. These filters iteratively update trajectory estimates as new data become available, effectively refining the positional accuracy over time (Choi and Christensen, 2013; Wang et al., 2019). Kalman filters employ a linear predictive model that is well suited for contexts with Gaussian noise, offering a clear framework for the prediction and correction of estimates. In contrast, particle filters provide a versatile nonlinear solution, maintaining a cloud of potential state hypotheses (“particles”) and updating their likelihoods in light of new observational data. This approach is particularly effective in handling arbitrary distributions of state variables, making it adaptable to a wide range of real-world conditions.

In addition, further improvements have been achieved by integrating diverse data sources through data assimilation techniques (Ali Arman and Tampère, 2022; Chen et al., 2022; Xie et al., 2018). These techniques merge information from traffic signals, loop detectors, and occasional GPS snapshots to construct more comprehensive trajectory estimates. For example, Chen et al. (2022) and Xie et al. (2018) demonstrated how integrating these heterogeneous data sources can significantly enhance the accuracy of trajectory reconstructions in scenarios characterized by low-upload rates.

2.3. Deep learning approaches

As the scale and complexity of trajectory data have increased, deep learning has emerged as a promising alternative to traditional probabilistic and heuristic-based models. Unlike conventional approaches, deep learning methods can learn complex spatial-temporal dependencies from large datasets, making them particularly effective in handling noisy and sparse GPS data, thereby facilitating accurate trajectory reconstruction.

One of the most commonly used architectures in trajectory reconstruction is the sequence-to-sequence (Seq2Seq) architecture, which was initially popularized by Sutskever et al. (2014) for natural language processing (NLP). Seq2Seq models map an input sequence to an output sequence of varying lengths, making them well suited for reconstructing trajectories from GPS data with low sampling rates. Traditional recurrent neural networks (RNNs), including long short-term memory (LSTM) and gated recurrent unit (GRU) models, have been widely adopted in trajectory reconstruction. For example, Wang et al. (2019) proposed a two-stage solution that initially recovers high-sample trajectories in free space, followed by a map matching algorithm, i.e., the HMM (Newson and Krumm, 2009), to pinpoint actual GPS locations on the map. While these models effectively capture sequential dependencies, they suffer from high computational cost or vanishing gradients for very long sequences, a common case when trajectories have extended durations or sparse observations.

To address these challenges, researchers have incorporated transformer-based architectures (Vaswani et al., 2017) and their variants, such as bidirectional encoder representations from transformers

(BERT), into trajectory reconstruction. Transformers introduce a self-attention mechanism, which enables parallelization over sequence elements (Bahdanau et al., 2016), improving efficiency and model performance, particularly in handling long-range dependencies (Devlin et al., 2018; Radford et al., 2018; Raffel et al., 2020). Xia et al. (2021) developed an attention-based model that explores both intra- and inter-trajectory mechanisms for modeling mobility regularities and patterns.

Moreover, task-by-task trajectory reconstruction often involves error propagation. In contrast, the end-to-end solutions proposed by Ren et al. (2021) and Chen et al. (2023), featuring a map-constrained decoder model, have demonstrated superior performance over two-stage methods. Despite these advancements, such data-driven approaches, which rely on deep learning, face interpretability challenges because of the opaque nature of neural networks. Ongoing research continues to refine these models with a focus on enhancing efficiency, interpretability, and adaptability across various scenarios (Liao et al., 2024; Zhang et al., 2023).

To further improve model robustness under sparse and noisy conditions, recent studies have increasingly employed data fusion strategies that integrate heterogeneous sources such as inertial measurement unit (IMU) signals, map semantics, and roadside sensor data. Such multimodal integration enhances resilience to signal degradation and contributes to greater spatial consistency, particularly in complex urban environments (Bai et al., 2024; Lin et al., 2025b). Concurrently, the incorporation of pretrained large language models (LLMs) based on transformer architectures has substantially improved model generalizability (Lan et al., 2024; Liu et al., 2023; Mahmud et al., 2025). By leveraging prior knowledge from large-scale sequential data, LLM-based approaches can adapt to previously unseen urban layouts or trajectory patterns without necessitating full model retraining, thereby offering a promising direction for scalable and transferable spatiotemporal modeling.

2.4. Summary

In summary, the field of trajectory reconstruction has evolved from traditional heuristic estimation to advanced deep learning models and optimization-based approaches, each contributing to more accurate and robust solutions in various application domains. However, existing models encounter significant challenges when handling sparse input data and ultralong target sequences, limiting their effectiveness in real-world scenarios where only intermittent GPS observations are available, and extended reconstructions are needed. To address these limitations, we propose a transformer-based architecture specifically designed for ultralong trajectory reconstruction. By segmenting the output sequence into manageable chunks and incorporating heuristic information, our approach effectively balances accuracy, computational efficiency, and scalability.

3. Methodology

In this section, we detail the proposed ProChunkFormer model, depicted in Fig. 2. Given a sparse and noisy trajectory, the input point

sequence is converted to a trajectory embedding and then fed into encoders. Inspired by the attention mechanism widely used in sequential problems, we adopt the renowned transformer structure, which uses multi-head attention to capture the global spatial-temporal features of trajectories. Then, the decoders first generate a map-matched semi-high-frequency sparse trajectory (MM-Semi) as a skeleton under the constraint of the road network by predicting the road segment identifier (ID) and moving ratio simultaneously for each timestep. The remainder of the trajectory is then divided into separate blocks, with each block generating the segment between two consecutive MM-Semi points. Finally, the complete target trajectory (MM-Full) is obtained by concatenating all the generated blocks.

3.1. Problem definition

A road network is represented as a directed graph $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = (v_1, v_2, \dots, v_M)$ denotes the set of nodes with size M and $\mathcal{E} = (e_1, e_2, \dots, e_N)$ denotes the set of edges with size N . Consider a raw sparse noisy trajectory $\mathcal{T} = \{p^0, p^1, p^2, \dots, p^n\}$, in which each point $p^i = (lat, lng, t)$ represents the latitude, longitude, and timestamp of the i -th GPS position obtained directly from a device. Here, superscript numbers denote the index of points in the raw trajectory. The raw GPS points can be projected onto the road network via map-matching algorithms (e.g., HMM), resulting in a map-matched trajectory $\widehat{\mathcal{T}} = \{l^0, l^1, l^2, \dots, l^m\}$, where $l^i = (lat, lng, e, r, t)$ represents map-matched trajectory points. Here, e represents the ID of the matched road segment, and the start and end positions of the road segment are denoted as $r^s = (lat, lng)$ and $r^e = (lat, lng)$, respectively. Fig. 3 shows an example of a raw trajectory point p^i matched to a point l^i that is located on a road segment. The moving ratio r denotes the fraction of the distance moved over the length of the road segment. Let lat , lng and t with superscripts denote the corresponding values of specific reference points. The match function between road segment e , raw point p^i and map-matched point l^i can be written as

$$\begin{cases} r = \frac{D1}{D3} \\ D3 = D2 \times \cos \alpha \\ lat^{p^i} = lat^{r^s} + r^i \times (lat^{r^e} - lat^{r^s}) \\ lng^{p^i} = lng^{r^s} + r^i \times (lng^{r^e} - lng^{r^s}) \\ t^{p^i} = t^i \end{cases} \quad (1)$$

where $D1$ denotes the length of the road segment, $D2$ represents the distance between the start node and the raw point, $D3$ represents the distance between the start node and the map-matched point, and α represents the deviation angle between the raw point and the road segment. Note that curved road segments should be divided into multiple approximately straight segments, allowing Eq. (1) to be applied effectively.

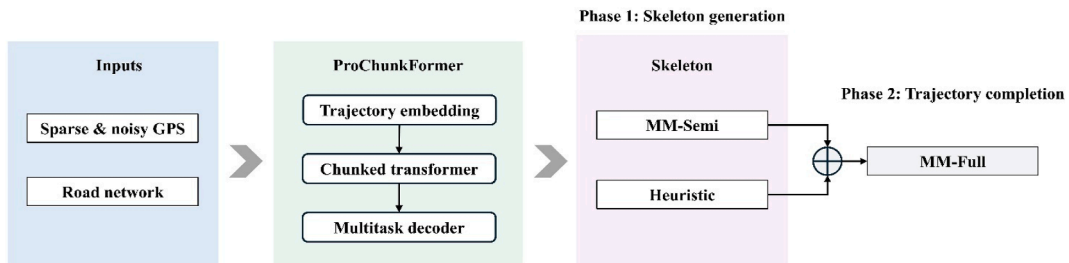


Fig. 2. Overview of the proposed ProChunkFormer framework.

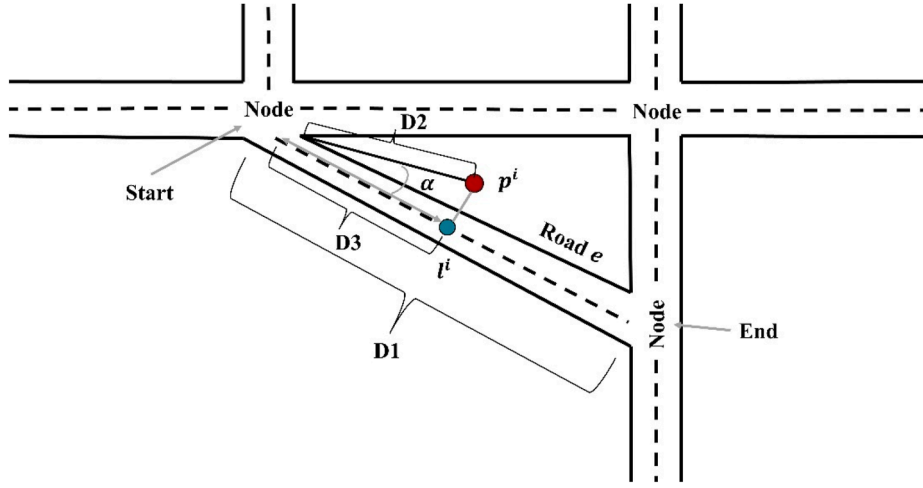


Fig. 3. Illustration of symbols for map matching.

Furthermore, the points in the raw trajectory $\mathcal{T}$ and map-matched trajectory $\hat{\mathcal{T}}$ are not necessarily uniformly sampled. Define a map-matched trajectory $\hat{\mathcal{T}}_\tau = \{l_0^0, l_1^1, l_2^2, \dots, l_T^T\}$, where the subscript denotes the timestep of each point. The points in $\hat{\mathcal{T}}_\tau$ are regularly sampled at a constant time interval, defining the sampling rate $\tau = (l_{i_r}^i - l_{i-1}^{i-1})/r$ as the time difference between two consecutive sample points. T is the total number of timesteps in trajectory $\hat{\mathcal{T}}_\tau$.

With these definitions, given a sparse and noisy GPS trajectory $\mathcal{T}$ (illustrated as black points in Fig. 4) with nonuniform time intervals and a target sampling rate τ that is sufficient for representing coherent travel trajectories, the objective of trajectory reconstruction is to obtain a map-matched trajectory $\hat{\mathcal{T}}_\tau$ (depicted as orange points in Fig. 4) with the specified uniform sampling rate τ .

3.2. ProChunkFormer

In this subsection, we present the details of the key modules in the proposed ProChunkFormer, including the trunked transformer architecture, the multitask learning decoder, and heuristic information integration.

3.2.1. Chunked transformer

Let k denote the number of chunks and b the number of steps in each chunk. The detailed structure of the proposed chunked transformer is illustrated in Fig. 5. The encoder in the chunked transformer follows the standard transformer architecture, as shown in the left section of Fig. 6. Let f_{TE} denote the function of the encoder module. Given the input embedding $\mathbf{Z}^e \in \mathbb{R}^{n \times d}$, where n is the length of the input trajectory and d is the embedding dimension, the output of the encoders can be expressed as $f_{TE}(\mathbf{H}^e)$. For a given raw trajectory $\mathcal{T}$, the initial input to the transformer encoder is the embedding of the raw trajectory combined with positional embedding as

$$\mathbf{H}^e = \mathbf{Z}^e + \text{PE}(\mathbf{Z}^e) \quad (2)$$

where $\mathbf{Z}^e$ is the embedding of the raw trajectory and $\text{PE}(\mathbf{Z}^e)$ represents the positional embedding, which can be obtained either through fixed sinusoidal functions or learnable embedding layers (Vaswani et al., 2017). The output memory of the encoder module is computed as

$$\text{Memory} = f_{TE}(\mathbf{H}^e) \quad (3)$$

The progressive decoding implementation is divided into two stages. First, the model generates the MM-Semi trajectory with a time interval of $b\tau$, where b denotes the number of steps in each chunk and τ

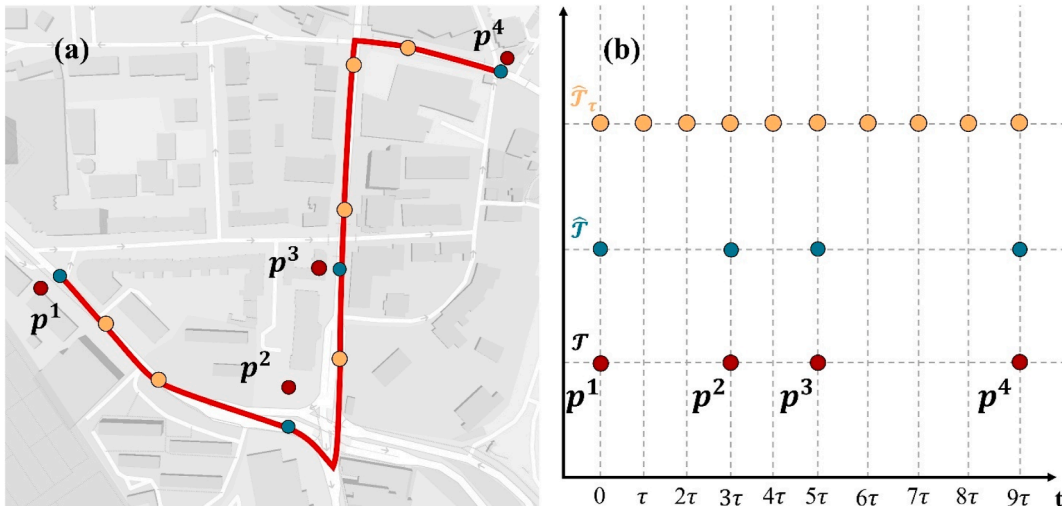


Fig. 4. Illustration of trajectory reconstruction. (a) Example of GPS trajectory points (p^1, p^2, p^3, p^4) on a map. (b) Sampling intervals for the raw trajectory (red), map-matched trajectory (blue), and reconstructed trajectory (orange). Map background © MapTiler. Based on OpenStreetMap data.

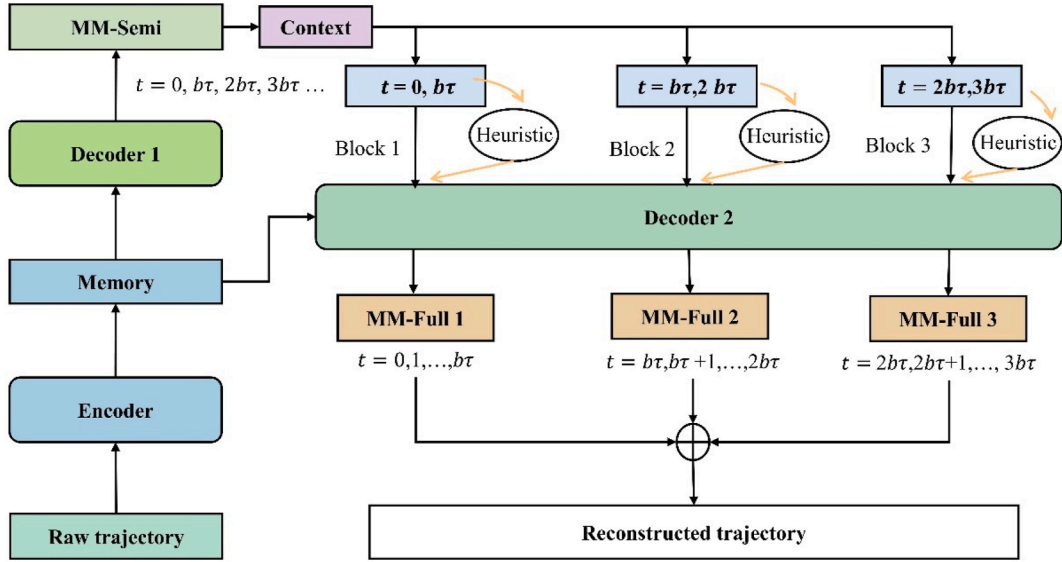


Fig. 5. Structural overview of the chunked transformer.

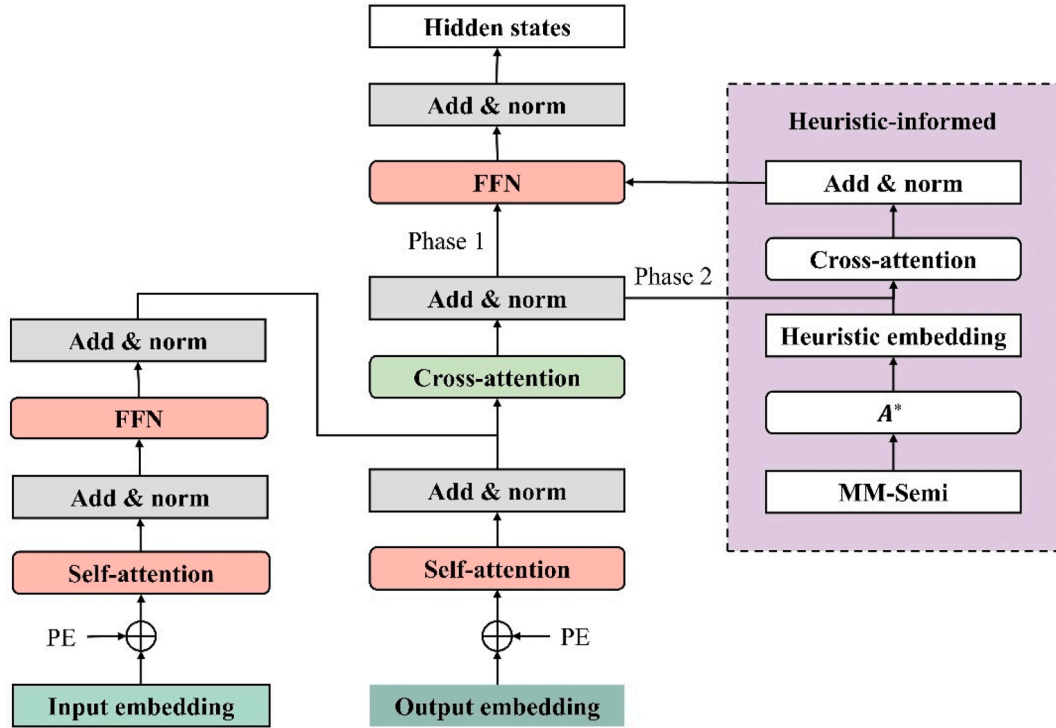


Fig. 6. Architecture of the proposed transformer-based computation graph. The left section illustrates the encoder structure, whereas the right section depicts the decoder structure. The dashed-line box highlights the additional influence of the shortest path from MM-Semi on the hidden state computation.

represents the time interval between consecutive points. Then, on the basis of the generated coarse-grained trajectory, it reconstructs intermediate points between every two consecutive points in parallel.

The input of the decoder in the first stage includes two parts: the *Memory* output from the encoder and the embeddings of previously generated trajectories, $\mathbf{Z}^d \in \mathbb{R}^{k \times d}$. Using f_{TD} as the function in Decoder 1, the hidden state $\mathbf{H}_{b\tau}$ for generating MM-Semi trajectory $\widehat{\mathcal{T}}_{b\tau}$ can be calculated as

$$\begin{cases} \mathbf{H}^d = \mathbf{Z}^d + \text{PE}(\mathbf{Z}^d) \\ \mathbf{H}_{b\tau} = f_{TD}(\text{Memory}, \mathbf{H}^d) \\ \text{s.t. } t = ib\tau, 0 \leq i < k \end{cases} \quad (4)$$

In the second stage, the fine-grained trajectory is reconstructed in parallel in the context of the MM-Semi trajectory. Let f_{TD} denote the function of Decoder 2. For the i -th blocks in the second stage, the hidden state output from Decoder 2 can be computed as

$$\begin{cases} \mathbf{H}_f^i = f_{TD}^i(\text{Memory}, \mathbf{H}_{br}, \mathbf{Z}^d) \\ \text{s.t. } i\tau \leq t \leq (i+1)\tau, 0 \leq i < k \end{cases} \quad (5)$$

By combining the results of all the blocks, the hidden states for all the time steps are represented as

$$\mathbf{H} = [\mathbf{H}_f^0, \mathbf{H}_f^1, \dots, \mathbf{H}_f^{k-1}] \quad (6)$$

3.2.2. Multitask learning

After the hidden state $\mathbf{H}$ is obtained from the chunked transformer module, a multitask decoder is used to reconstruct the trajectory. We predict the road segment ID e via a Softmax function and infer the moving ratio r via a sigmoid function (Fig. 7). The map-matched trajectory can be generated via the well-trained multitask decoder:

$$P_{l_t, e|\theta} = \text{Softmax}(\mathbf{H}) \quad (7)$$

$$\mathbf{h} = \text{Embedding}(l_t, e) \quad (8)$$

$$P_{l_t, r|l_t, e, \theta} = \text{Sigmoid}(\mathbf{H} \oplus \mathbf{h}) \quad (9)$$

where $P_{\cdot|\theta}$ denotes the predicted probability determined by the model with parameter θ , and l_t represents the predicted map-matched points for time step t . $\text{Embedding}(\cdot)$ is an embedding layer for the road ID. During the training process, the model concurrently predicts the road segment e and moving ratio r , enabling us to identify the exact location on the road network at each timestep, denoted by τ .

To predict the road segment ID, we employ cross-entropy loss for multiclassification, which quantifies the similarity between the ground truth and the predicted trajectories:

$$L_{id} = \frac{1}{k} \sum_{j=1}^T \sum_{i=1}^N y_{i,j} \log(P_{l_j, e=e_i|\theta}) \quad (10)$$

$$y_{i,j} = \begin{cases} 1 & \text{if } l_j, e = e_i \\ 0 & \text{otherwise} \end{cases} \quad (11)$$

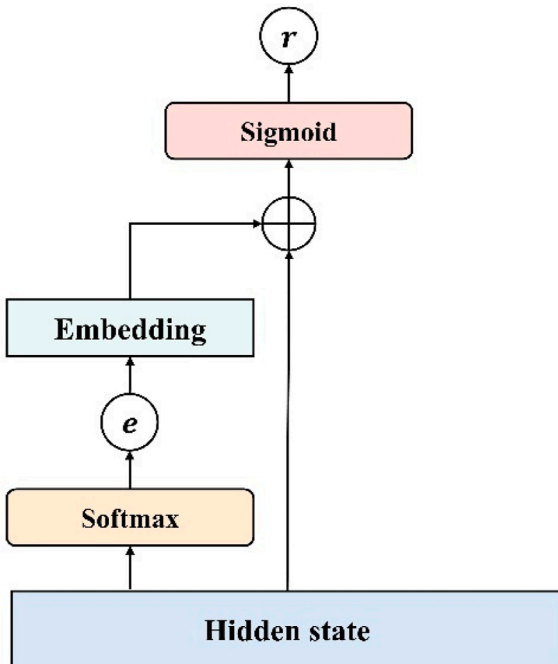


Fig. 7. Multitask learning for trajectory reconstruction.

where T is the duration of the reconstructed trajectory $\widehat{\mathcal{T}}_{br}$ in units of time step τ and N represents the total number of edges in the graph as previously mentioned.

Instead of employing a mean squared loss function directly for the moving ratio, which may become invalid when the prediction of the road segment is incorrect, we utilize the root mean square error (RMSE) to measure the distance between the ground truth and the predicted points as our loss function:

$$L_{\text{dist}} = \sqrt{\frac{1}{n} \sum_{j=1}^n \text{dist}(l_j, l_j')^2} \quad (12)$$

Overall, the total training loss combines these two loss functions:

$$L = L_{id} + \lambda L_{\text{rate}} \quad (13)$$

where λ is a hyperparameter used to balance the two losses in our model.

3.3. Chunked transformer and autoregressive transformer

In the validation process, Seq2Seq models typically utilize autoregressive decoding (AD) to generate the target sequence on the basis of previously generated tokens:

$$P_{\widehat{\mathcal{T}}_\tau} = \prod_{t=0}^T P_{l_t|l_0, l_1, l_2, \dots, l_{t-1}} \quad (14)$$

In this study, we adopt chunked processing (CP) for progressively reconstructing trajectories. Instead of generating the target trajectory step by step, we employ a two-stage method. In the first stage, we generate an MM-Semi trajectory $\widehat{\mathcal{T}}_{br} = \{l_0, l_{br}, l_{2br}, \dots, l_{kbr}\}$ with a time interval of br . In the second stage, this coarse-grained trajectory serves as the constraint context for generating the remaining trajectory. On the basis of the number of points generated in the first stage, the second stage is divided into k blocks. For the i -th block, the algorithm generates trajectory points between l_{i-1} and l_i . The size of each block $b = (l_i - l_{i-1})/\tau$ is calculated as the time step of l_{i-1} and l_i in units of the target sampling rate τ . The processes can be written as

$$P_{\widehat{\mathcal{T}}_{br}} = \prod_{t=1}^k P_{l_t|l_0, l_{br}, l_{2br}, \dots, l_{(t-1)br}} \quad (15)$$

$$P(\widehat{\mathcal{T}}_\tau) = P_{\widehat{\mathcal{T}}_{br}} \prod_{i=0}^{k-1} \prod_{j=ibr+1}^{(i+1)br-1} P_{l_j|l_{ibr+1}, l_{ibr+2}, \dots, l_{j-1}} \quad (16)$$

Tables 1 and 2 present the decoding procedures of the autoregressive decoder and the proposed chunked decoder, respectively. One of the advantages of CP lies in its ability to generate the remaining points for each block in parallel after decoding the coarse-grained trajectory in the first stage. This approach significantly enhances model efficiency, particularly when dealing with large target sequences, without compromising performance. Given the target trajectory $\widehat{\mathcal{T}}_\tau$ with a total of T timesteps, the complexity of the multi-head attention module in AD is $O(T^2)$, representing the primary computational cost of the decoder. For the CP, the complexity of the first stage is $O(k^2)$. In the second stage,

Table 1

Auto-regressive decoding process for trajectory reconstruction.

Input: $\text{Memory}, \mathbf{Z}^d$
1 Max time step T , start token S , end token E
2 Initial input $[S]$, time step $i = 0$
3 While $i \leq T$ and current prediction $\hat{l}_{it}$ is not E do:
4 Current time prediction $\hat{l}_{it}$
5 Add to $\widehat{\mathcal{T}}_\tau$
6 $i \leftarrow i + 1$
Output: Recovered trajectory $\widehat{\mathcal{T}}_\tau$

Table 2

Chunked decoding process for trajectory reconstruction.

Input: Memory, Z^d
1 Max time step T , number of blocks k start token S , end token E
2 Initial input S , block ID $i = 0$
3 While $i \leq k$ and current prediction l_{br}^i is not E do
4 Current time prediction l_{br}^i
5 Add to $\widehat{\mathcal{F}}_{br}$
6 $i \leftarrow i + 1$
7 Got $\widehat{\mathcal{F}}_{br}$
8 Parallel computing:
9 For block i :
10 Timestep $j = ibr + 1$
11 While $j \leq (i+1)br + 1$ and current prediction l_j^i is not E do:
12 Current prediction l_j^i
13 Add to $\widehat{\mathcal{F}}_i$
14 $j \leftarrow j + 1$
Output: Recovered trajectory $\widehat{\mathcal{F}}_i$

each block has a complexity of $O(b^2)$, so the total complexity is $O(k^2 + b^2)$.

To compare $O(T^2)$ and $O(k^2 + b^2)$ under the constraints of $kb = T$ and $T > k > 1$, we calculate when $O(k^2 + b^2)$ is larger than (T^2) :

$$\begin{cases} k^2 + b^2 - T^2 > 0 \\ \frac{T^2}{k^2} + k^2 - T^2 > 0 \\ T^2 \left(1 - \frac{1}{k^2}\right) < k^2 \\ T^2 < \frac{k^4}{k^2 - 1} \approx k^2 \end{cases} \quad (17)$$

According to the calculations, $O(k^2 + b^2)$ is always smaller than $O(T^2)$. This finding demonstrates that when $T > k > 1$, $O(k^2 + b^2 < O(T^2))$ always holds true.

3.4. Heuristic enhanced trajectory reconstruction

Given a raw trajectory $\mathcal{F} = \{p^0, p^1, p^2, \dots, p^n\}$, after the first step of the CP approach, we obtain the MM-Semi trajectory $\widehat{\mathcal{F}}_{br} = \{l_{br}^0, l_{br}^1, \dots, l_{br}^k\}$ from Eq. (16). To improve the model performance over long sampling intervals, we make use of the shortest path between every two consecutive points from $\widehat{\mathcal{F}}_{br}$ as prior information to the trajectory generation of each block. To do that, for each pair of consecutive points $l_{(i-1)br}^{i-1}$ and l_{ibr}^i , we find the shortest path between them over graph G via the A-star algorithm, which results in the local shortest path $A_{i-1,i} = (e_{i-1,i}^1, e_{i-1,i}^2, \dots, e_{i-1,i}^q)$, where $e_{i-1,i}^j$ denotes the j -th segment ID of the shortest path between points $l_{(i-1)br}^{i-1}$ and l_{ibr}^i , $e_{i-1,i}^1 = l_{i-1}.e$ and $e_{i-1,i}^q = l_i.e$, and q is the length of $A_{i-1,i}$. The contribution of the heuristic shortest path is considered by a cross-attention layer, as shown in Fig. 6. The attention value with heuristic information can be modified as follows:

$$h_{i-1,i} = \text{Embedding}(A_{i-1,i}) \quad (18)$$

$$H_f^i = f'_{TD}(\text{Memory}, H_{br}, h_{i-1,i}, Z^d) \quad (19)$$

Notably, other heuristic information can also be incorporated in a similar manner, although we have only implemented it for the shortest path.

4. Case study

4.1. Experimental settings

We validate the effectiveness of our model via a real-world taxi GPS dataset, which inherently contains noise due to factors such as signal interference, environmental obstructions, and device inaccuracies. The road network is obtained from OpenStreetMap. The full dataset contains trajectories of 999,082 GPS records in Porto, Portugal, collected over a period of one year from 2013 to 2014. For evaluation purposes, we focus on a specific area in Porto, with latitudes ranging from 41.149 to 41.154 and longitudes ranging from -8.634 to -8.629 . After data cleaning (Zheng, 2015), the dataset consists of 120,116 trajectories continuously sampled every 15 s. The selected area includes 263 road segments, which correspond to the output dimensionality of the ID prediction module.

Fig. 8 illustrates the process of generating training data for our model. The original noisy GPS data are converted to target map-matched trajectories via HMM-based methods, which we refer to as reconstructed trajectories. Prior research has shown that with a sampling interval of 15 s, the map-matching algorithm can achieve an accuracy of up to 99%, indicating that the matched results are sufficiently accurate to serve as ground truth (Newson and Krumm, 2009). To simulate sparse conditions, we down sample the raw GPS data by a certain proportion, and these sparser noisy trajectories are used as inputs for our model. The dataset is then split into training, validation, and testing sets at a ratio of 7:2:1 following standard machine learning practices to ensure robust model evaluation.

The experiments are conducted on a GPU server equipped with 3 Quadro GV100 GPUs, each with 32 GB of memory. The ProChunkFormer model consists of two layers of encoder and decoder components, with the dimension of the hidden layers set to 512. The block size k is set to 8. Following previous work (Ren et al., 2021), the multitask weight parameter λ is set to 10. The training batch size is 64, with a learning rate of 0.0001, and the model is trained for 30 epochs. Notably, owing to limitations in computational resources, the maximum length of the GPS data was set to 64. GPS data exceeding this length were divided into smaller sequences.

4.2. Evaluation metrics

In this study, both the accuracy of road segment reconstruction and the error of location inference are calculated to evaluate the performance of our model. To evaluate the accuracy of route reconstruction, we compare the recovered road segments $\mathcal{E}_R$ with the ground truth $\mathcal{E}_G$. Following previous work (Cui et al., 2018; Kurashima et al., 2010; Ren et al., 2021), the recall, precision, and F1 score (F1) are defined as

$$\text{recall} = \frac{\mathcal{E}_R \cap \mathcal{E}_G}{\mathcal{E}_G} \quad (20)$$

$$\text{precision} = \frac{\mathcal{E}_R \cap \mathcal{E}_G}{\mathcal{E}_R} \quad (21)$$

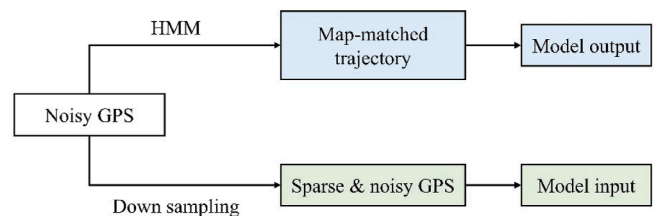


Fig. 8. Generation progress of training data.

$$F1 = \frac{2 \times \text{recall} \times \text{precision}}{\text{recall} + \text{precision}} \quad (22)$$

To measure the error in location inference, we adopt two types of mean absolute error (MAE) metrics: the point-wise MAE based on Euclidean distance, and MAE_RN based on road network path distance. Both are calculated between corresponding GPS points p and p' :

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\text{dist}(p, p')| \quad (23)$$

$$\text{MAE_RN} = \frac{1}{n} \sum_{i=1}^n |\text{dist}_m(p, p')| \quad (24)$$

where $\text{dist}(p, p')$ denotes the Euclidean distance between points p and p' , and $\text{dist}_m(p, p')$ represents the path length between the two points along the road network.

4.3. Baseline

We benchmark our model's performance against three baselines, namely, "Linear + HMM" (Hoteit et al., 2014), "MTrajRec" (Ren et al., 2021), and the standard transformer, which includes machine learning and deep learning approaches. The first method uses linear interpolation to generate a high-sample trajectory, followed by an HMM algorithm to obtain the map-matched trajectory under the assumption that trajectories move in a straight line at a uniform speed. The second method is a recently published state-of-the-art deep learning method for trajectory reconstruction in an end-to-end manner. The third model is a traditional autoregressive decoding transformer with multitask learning designed to highlight the effectiveness of our proposed progressive architecture.

Furthermore, we also evaluated the proposed model in five distinct configurations for the purpose of ablation study: (1) basic chunked transformer, referred to as "ProChunkFormer"; (2) chunked transformer without full context in the second stage, considering only start and end points, denoted as "ProChunkFormer (SE)"; (3) ProChunkFormer (SE) with Encoder 1 and Encoder 2 sharing the same multi-head attention module, termed "ProChunkFormer (SE + Share)"; (4) ProChunkFormer with Encoder 1 and Encoder 2 sharing the same multi-head attention module, referred to as "ProChunkFormer (Share)"; and (5) ProChunkFormer with heuristic information, labeled "ProChunkFormer (Heuristic)".

4.4. Results

4.4.1. Overall performance

The comparative experimental results are shown in Table 3. Our model was evaluated against baseline models across several metrics, including accuracy, F1 score, MAE, and RN_MAE. The evaluation was conducted on sparse and noisy trajectories sampled at intervals of 60, 120, and 240 s, corresponding to retention rates of 25%, 12.5%, and 6.25%, respectively, of the points in the raw GPS data, which were originally sampled at an interval of 15 s.

As expected, due to the increase in uncertainty, the performance of all the models deteriorates as the sampling interval increases. Notably, the learning-based models MTrajRec and transformer outperform the HMM-based two-stage models. The transformer model generally outperforms MTrajRec, but the performance difference is marginal. The results also indicate that ProChunkFormer achieves performance comparable to that of the autoregressive transformer. At sampling intervals of 60 and 120 s, the autoregressive transformer method slightly outperforms ProChunkFormer in most scenarios, with only a few cases where ProChunkFormer performs better. However, at a sampling interval of 240 s, ProChunkFormer achieves a significant performance gain over the conventional transformer baseline. Specifically, ProChunkFormer delivers a 23.1% relative improvement in accuracy, an 18.6% increase in the F1 score, a 22.3% reduction in the MAE, and a 25.1% decrease in the MAE_RN.

The results of the ablation study are presented in Table 4. Experiments with ProChunkFormer (SE), which uses less context information, show a marginal drop in performance in most cases, although some evaluation metrics show slight improvements in rare instances. The results from ProChunkFormer (Share), which shares the same decoder module across two stages for enhanced efficiency, show a minor performance cost, particularly at the sampling interval of 240 s. However, applying decoder sharing to ProChunkFormer (SE) results in more significant performance degradation.

The results of ProChunkFormer (Heuristic) highlight the value of incorporating heuristic information in the trajectory reconstruction task. Incorporating heuristics enhances performance, particularly at a sampling interval of 240 s, although it results in slight performance declines in certain cases. To further assess the impact of heuristic information on the predictions of the trajectory reconstruction model, we visualized the total loss on the validation dataset in Fig. 9. The blue line represents the total loss of ProChunkFormer, whereas the orange line denotes the total loss of ProChunkFormer (Heuristic). As illustrated, the model with heuristic information exhibits faster loss reduction in the early stages of training. However, as training progresses, both models converge to similar levels of total loss.

4.4.2. Running efficiency

To assess the efficiency of the proposed model, we compared the running times of several models, including Linear + HMM, MTrajRec, Transformer, and ProChunkFormer. The experiments were carried out with sampling intervals of 60, 120, and 240 s, and the results are presented in Fig. 10. As shown, Linear + HMM is significantly slower than the other learning-based models are, whereas MTrajRec requires the least amount of running time under our configuration. When comparing the transformer with ProChunkFormer at sampling intervals of 60, 120, and 240 s, the running time was reduced by 13.9%, 16.5%, and 20.7%, respectively. Additionally, as the sampling intervals increased, the running time of all the models decreased. For the Linear + HMM model, this is because the linear interpolation method needs to process fewer points. In contrast, for learning-based models, the decoder complexity remains constant, as it is influenced only by the target sampling interval,

Table 3

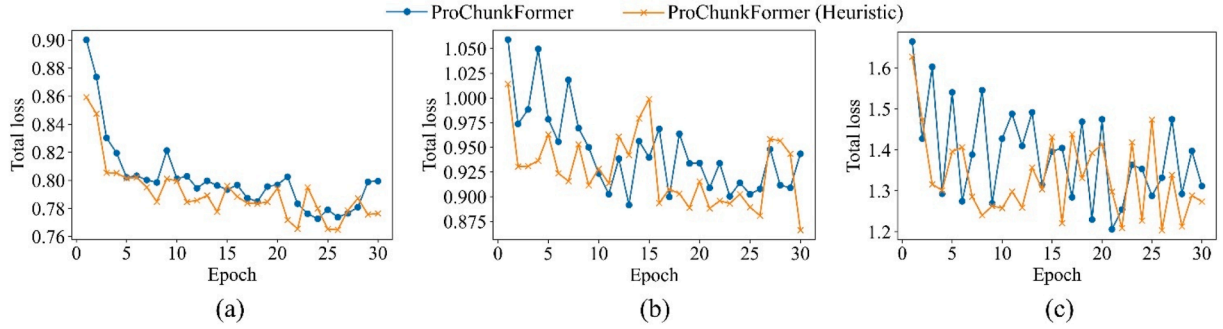
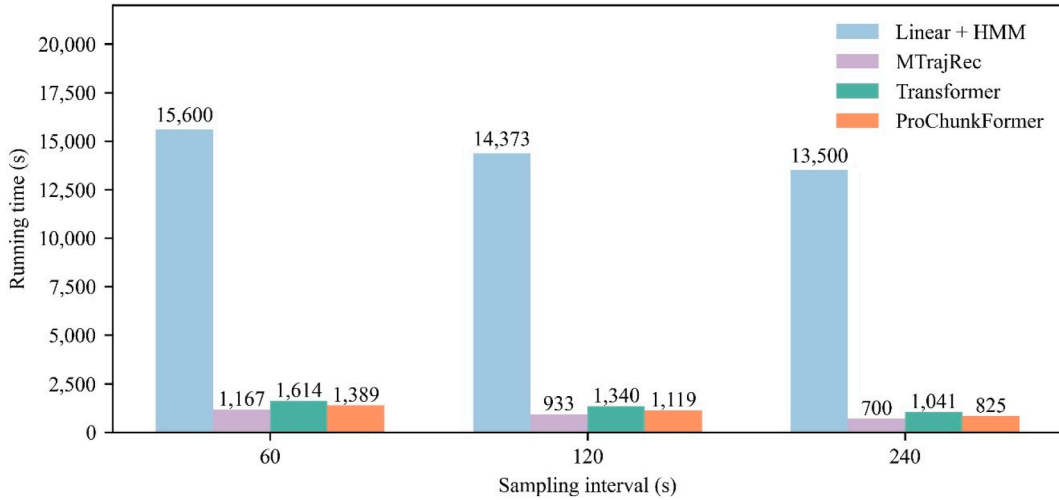
Results of different methods. Each intervening row shows the performance difference between the method below and the one above, with red highlighting a positive change and green indicating a negative change.

Methods	Accuracy			F1 Score			MAE			MAE_RN		
	240	120	60	240	120	60	240	120	60	240	120	60
Linear + HMM	0.3154	0.3392	0.4229	0.4515	0.5104	0.5684	91.98	80.36	54.71	199.31	179.24	141.98
	+7.4%	+37.1%	+30.7%	+6.8%	+26.2%	+17.4%	-18.8%	-39.6%	-35.0%	+3.5%	-42.2%	-44.4%
MTrajRec	0.3387	0.4650	0.5527	0.4823	0.6441	0.6674	74.70	48.57	35.56	206.30	103.53	78.94
	-4.5%	+1.4%	+1.4%	-9.8%	+0.4%	+0.8%	+7.2%	+0.8%	-3.3%	-13.0%	-3.7%	-2.6%
Transformer	0.3236	0.4714	0.5606	0.4349	0.6468	0.6725	80.09	48.17	34.40	179.39	99.70	76.92
	+23.1%	+0.2%	-1.9%	+18.6%	-0.3%	-1.2%	-22.3%	+0.6%	+6.0%	-25.1%	-1.9%	+5.2%
ProChunkFormer	0.3982	0.4723	0.5497	0.5158	0.6447	0.6643	62.20	48.47	36.47	134.43	97.78	80.95

Table 4

Results of ProChunkFormer with different variations.

Methods	Accuracy			F1 Score			MAE			MAE_RN		
	240	120	60	240	120	60	240	120	60	240	120	60
ProChunkFormer	0.3982	0.4723	0.5497	0.5158	0.6447	0.6643	62.20	48.47	36.47	134.43	97.78	80.95
ProChunkFormer (SE)	0.4063	0.4701	0.5589	0.5151	0.6410	0.6717	62.48	49.88	35.34	132.44	99.08	77.18
ProChunkFormer (SE + Shared)	0.3380	0.4192	0.5415	0.3968	0.5794	0.6498	87.54	66.11	41.66	174.44	123.20	87.14
ProChunkFormer (Shared)	0.3895	0.4725	0.5511	0.5025	0.6466	0.6662	65.31	48.09	36.82	142.62	95.92	80.72
ProChunkFormer (Heuristic)	0.4022	0.4657	0.5601	0.5245	0.6456	0.6708	60.68	48.03	34.79	136.76	96.40	76.77

**Fig. 9.** Total validation loss for ProChunkFormer and ProChunkFormer (Heuristic): (a) 60-s intervals, (b) 120-s intervals, and (c) 240-s intervals.**Fig. 10.** Running time of different models under varying sampling intervals.

which does not change. However, the encoder complexity decreases as the sampling intervals of the input data increase. As a result, the overall runtime decreases with increasing sampling interval.

4.4.3. Block hyperparameters

Our model introduces two important hyperparameters: the number of blocks k and the block size b . Given a specific target sequence length, the number of blocks is inversely proportional to the block size:

$$b \propto \frac{1}{k} \quad (25)$$

We carried out experiments for a set of block sizes [2, 4, 6, 8, 10, 12, 14, 16, 18, 20], as shown in Fig. 11. The blue, orange, and green lines correspond to sampling intervals of 60, 120, and 240 s, respectively. For accuracy, the results are relatively stable across different block sizes when the sampling interval is 60 or 120 s. However, with a sampling interval of 240 s, the accuracy tends to decline as the block size

increases. In terms of the F1 score, performance improves as the block size increases. For the MAE, the results are stable when the sampling interval is 120 s. However, with a sampling interval of 240 s, the MAE increases significantly as the block size increases, whereas at a 60-s sampling interval, the MAE slightly decreases as the block size increases. For MAE RN, the results remain stable at sampling intervals of 60 and 120 s, but MAE RN tends to increase as the block size increases at a sampling interval of 240 s. Note that the results are averaged over the five epochs after the model converges to ensure stability.

The running efficiency of the two stages in CP is closely tied to the block configuration. Fig. 12 shows the average running time per epoch for different sampling intervals. As the block size increases, the running time initially decreases but then increases. For a 60-s sampling interval, the model is most efficient, with a block size of 14. For a 120-s sampling interval, the optimal block size is 18, whereas for a 240-s sampling interval, the model achieves maximum efficiency with a block size of 12.

5. Discussion

5.1. Effectiveness and efficiency of chunked transformer

This study investigates trajectory reconstruction via a transformer architecture and proposes a chunked decoder to increase model effectiveness for long-duration and long-interval trajectories. The experimental results for various configurations show that ProChunkFormer achieves a good balance between effectiveness and efficiency.

To evaluate the performance of chunk processing, we carry out experiments for both the autoregressive decoder and chunked decoder across different sampling intervals. While the overall performance of the AD is superior in most cases, the CP still performs effectively, particularly at lower sampling intervals. The potential reason for the edge of AD is that each trajectory block is decoded independently in the CP, slightly compromising performance. We implemented cross-stage attention to mitigate error propagation between stages. Nevertheless, it did not significantly enhance performance, possibly because the input of the second stage already includes information from the first stage. This suggests that CP is nearly as effective as AD, with CP even outperforming AD at a sampling interval of 240 s. Fig. 13 visualizes the cumulative total loss over time steps for the validation dataset, showing that CP accumulates errors more slowly than AD does, which may explain its superior performance in long sampling interval scenarios by mitigating error accumulation during autoregressive decoding.

The running time comparison in Fig. 10 demonstrates the efficiency of the proposed CP over AD, with CP consistently having shorter run-times across all sampling intervals. Additionally, the results highlight that learning-based models are significantly more efficient than HMM-based two-stage models. This is due to the large state spaces and long sequences, which greatly increase the complexity of the HMM method, whereas deep learning models benefit from GPU acceleration and parallelism, making them highly efficient in our experiments.

As the two stages of CP are both targeted at generating map-matched trajectories at different time intervals, they are potentially able to share the decoder to save computational resources. This can have a negative effect on the model's performance. As shown in Table 1, when we use a shared transformer decoder in the CP, the performance decreases significantly. This performance drop can be partially offset by incorporating cross-stage attention, which is less expensive than a full decoder module, indicating that the CP method can utilize a shared transformer decoder to improve efficiency.

The block characteristics are determined by the block size and the number of blocks. Fig. 11 shows that these hyperparameters can significantly affect training efficiency. The optimal block configuration varies depending on the sampling interval time. Fig. 10 shows that the performance of the model is multifaceted. In the case of long interval times, the model performance remains stable across block configurations for different evaluation metrics. However, at a sampling interval of 240 s, the F1 score, MAE, and MAE_RN increase, whereas the accuracy decreases, indicating less stability than shorter sampling intervals. Despite this, the decline in some metrics is compensated for by improvements in others, which make the overall performance as stable as in the case of shorter sampling intervals.

Heuristic information is expected to guide the generation of the target sequence. As validated in Fig. 9, this finding proves that it is useful in the early stage of training, increasing the convergence speed of the model. However, it does not reduce the overall total loss at convergence, likely because road users do not always follow the shortest path, introducing noise that can sometimes degrade model performance.

5.2. Error accumulation

Error accumulation is a critical issue in sequence generation tasks, particularly in models that rely on previously generated outputs as inputs for subsequent predictions. Both autoregressive transformers and

chunked transformers exhibit this phenomenon, but the underlying mechanisms differ due to their structural variations.

To generate the target sequence $\widehat{\mathcal{T}} = \{\hat{l}_0^0, \hat{l}_1^1, \hat{l}_2^2, \dots, \hat{l}_T^T\}$, the error at step i , denoted as ϵ^i , depends on the cumulative errors from the previous steps. An autoregressive decoder can be expressed as

$$\epsilon^i = f(\epsilon^0, \epsilon^1, \dots, \epsilon^{i-1}) \quad (26)$$

Errors made at earlier steps are fed into the model as inputs for later steps, causing subsequent predictions to deviate further from the true sequence. This recursive feedback loop leads to a snowball effect, especially for long sequences. The longer the sequence is, the more significant the cumulative error becomes. As the model progresses through the sequence, small inaccuracies in early predictions can escalate, resulting in a substantial deviation in the later stages.

Chunked transformers address the scalability issue of long sequences by dividing the sequence into smaller, independently processed chunks. For the j -th block, the error at step i is given by:

$$\epsilon^i = f(\epsilon_{jbr}^0, \epsilon_{jbr+1}^1, \dots, \epsilon_{jbr+i-1}^{i-1}) \quad (27)$$

This structural modification changes the error accumulation dynamics. Within each chunk, predictions remain autoregressive, leading to local error accumulation similar to that of the standard autoregressive transformer. However, errors are generally confined to the chunk level rather than propagating across the entire sequence.

Fig. 13 provides a visual comparison of the constructed trajectory using ProChunkFormer and the standard transformer. Both models achieve good results for points within Block 1. In contrast, ProChunkFormer continues to generate accurate predictions for Block 2, as chunk-level independence mitigates global error accumulation.

Nevertheless, chunk processing introduces new challenges, particularly at block boundaries. Since points within each chunk are generated dependently, errors near the edges of chunks can increase significantly. This phenomenon is evident in Fig. 14, which shows the mean absolute error (MAE) overall test samples for each timestep. Although ProChunkFormer achieves a lower overall MAE than the standard transformer does, error fluctuations are observed—particularly near block boundaries—with a block size of 8 in this experiment. Incorporating heuristic information effectively mitigates these fluctuations by providing additional contextual cues, thereby enhancing trajectory stability.

In summary, while ProChunkFormer effectively limits global error accumulation compared with standard transformers, its performance near chunk boundaries requires further optimization. This trade-off between local and global error dynamics highlights the importance of designing robust mechanisms for interchunk information sharing and continuity.

5.3. Model complexity for long-period trajectories

In the trajectory reconstruction problem, the goal is to reconstruct a densely sampled trajectory from a sparsely sampled trajectory. Since the output sequence length often greatly exceeds the input length, the computational complexity of the decoder becomes a critical factor for the practical application of trajectory reconstruction models. For transformer-based methods, the complexity of the decoder during training and inference (or prediction) differs significantly owing to differences in how computations are parallelized and the autoregressive nature of sequence generation during inference. Table 5 summarizes the space and time complexity of the model when the target sequence length is much larger than the number of blocks (i.e., $T \gg k$).

During training, the transformer decoder typically processes the entire sequence in parallel. All the time steps are computed simultaneously. The time and space complexity are both dominated by the self-attention mechanism, resulting in a complexity of $O(T^2)$. This quadratic

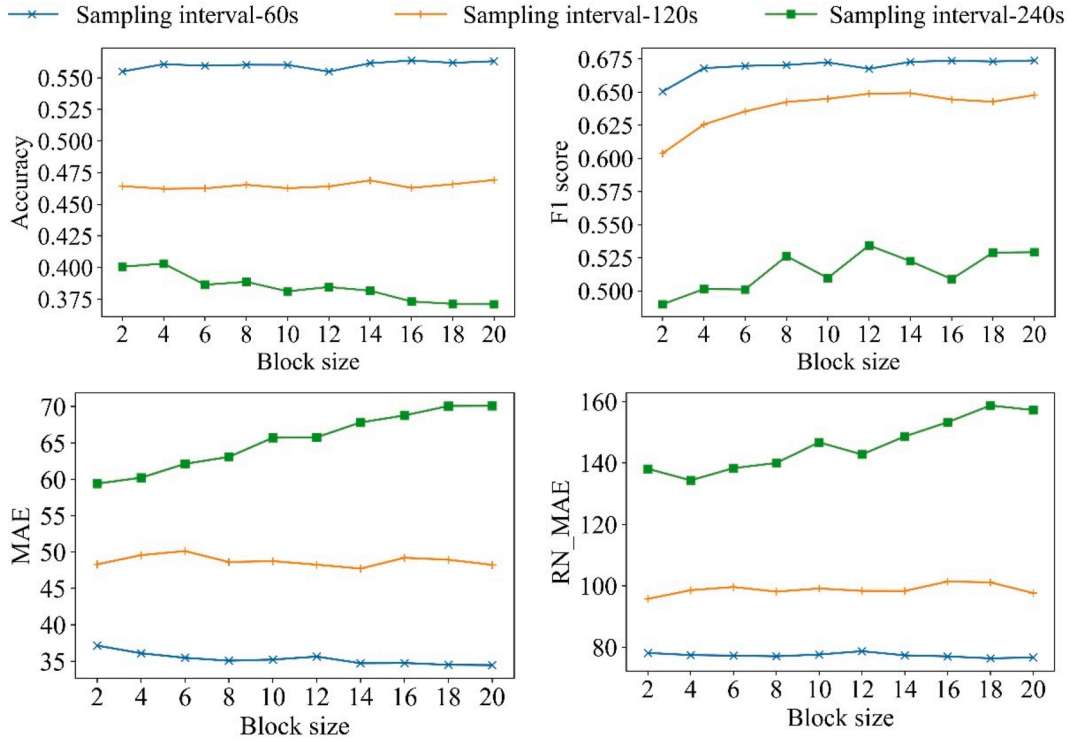


Fig. 11. Performance of ProChunkFormer for different block sizes.

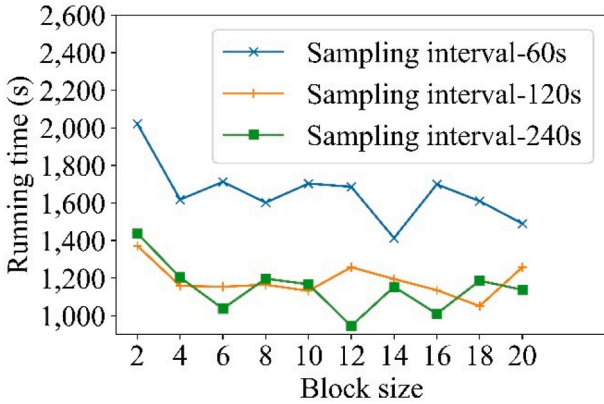


Fig. 12. Running time of ProChunkFormer for different block sizes.

dependency on the sequence length stems from the need to compute pairwise relationships for every token in the sequence. In contrast, during inference, the decoder generates the sequence autoregressively, one step at a time. At each step t , the self-attention mechanism computes relationships between all preceding t positions. Generating the entire sequence requires summing the complexities over all steps, resulting in a total complexity of $O(1^2 + 2^2 + \dots + T^2) \approx O(T^3)$. This cubic complexity makes inference significantly more computationally intensive than training, especially for long sequences.

For the training of a chunked transformer, the decoder's time and space complexity is reduced to $O(T^2/k)$ because of processing within individual blocks. Additionally, by leveraging parallel computation across blocks, the complexity can be further reduced to $O(T^2/k^2)$. During inference, the progressive decoding structure reduces the complexity from $O(T^3)$ in standard transformers to $O(T^3/k^3)$, offering substantial computational savings. These optimizations make the

chunked transformer highly scalable and efficient for trajectory reconstruction tasks with lengthy target sequences.

5.4. Limitations and future work

This work presents a novel chunked transformer for trajectory reconstruction, demonstrating both efficiency and strong performance. However, several limitations should be addressed. First, the grid representations and road segment embeddings over the map are learned via data-driven models, which may be vulnerable to data sparsity issues. The incorporation of pretrained embeddings for the grids and road segments could enhance the model's performance. Second, our investigation focuses primarily on the heuristic information of the shortest path for trajectory reconstruction. While this approach accelerates convergence, it does not significantly improve performance. A more in-depth exploration of various heuristic information types and integration methods for incorporating them into the model to guide prediction is necessary. Third, although the results show that CP is more efficient than AD is, it remains less efficient than MTrajRec is. An optimized transformer, such as sparse attention (Tay et al., 2020) or linear attention (Yang et al., 2023), could be employed to further improve model efficiency.

Although chunked transformers mitigate the propagation of global errors, they also introduce challenges at chunk boundaries, notably regarding continuity and consistency. While the integration of heuristic information can help alleviate these issues, exploring alternative methods for achieving higher-quality trajectory reconstruction remains valuable (Bahdanau et al., 2016; Beltagy et al., 2020; Dai et al., 2019). Moreover, even though the recovered trajectory is confined to the map, it may still lack continuity, which could hinder its practical application. Future research should therefore focus on developing approaches that generate continuous, map-constrained trajectories. Additionally, the chunked architecture has potential for enhancing the efficiency of other Seq2Seq models.

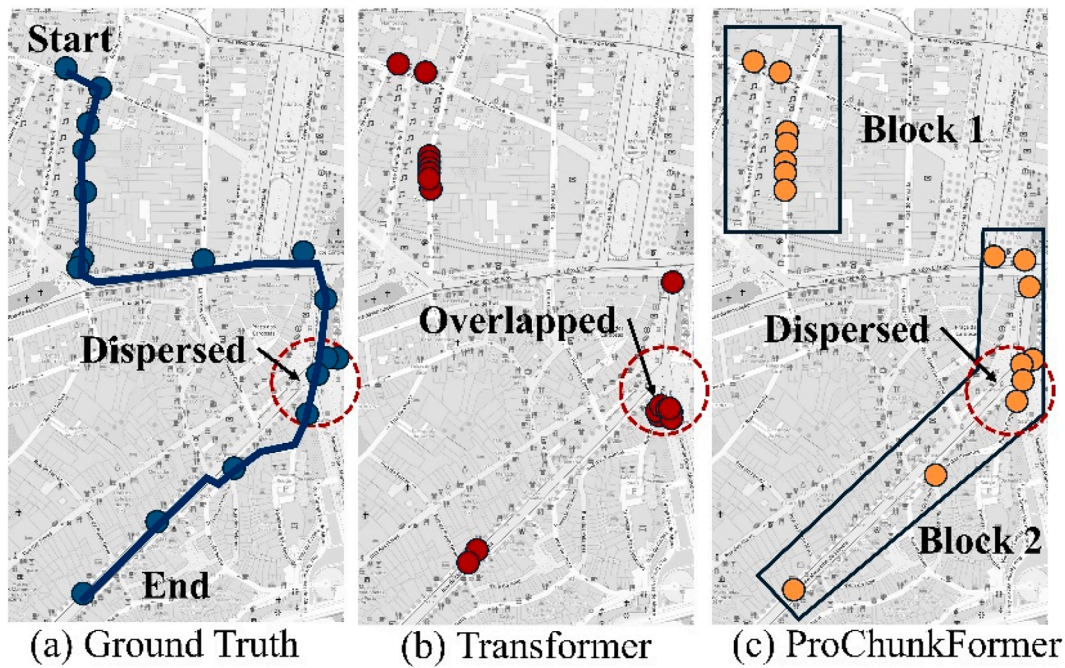


Fig. 13. Visualization of the reconstructed trajectory via Transformer and ProChunkFormer: (a) ground truth GPS points and the corresponding route; (b) GPS points reconstructed by the transformer model, where points within the red circle are densely clustered and overlapping; and (c) GPS points reconstructed by ProChunkFormer, segmented into Block 1 and Block 2. In contrast to those of the transformer, the points in the red circle are dispersed. Map data © OpenStreetMap contributors, licensed under CC BY-SA 2.0.

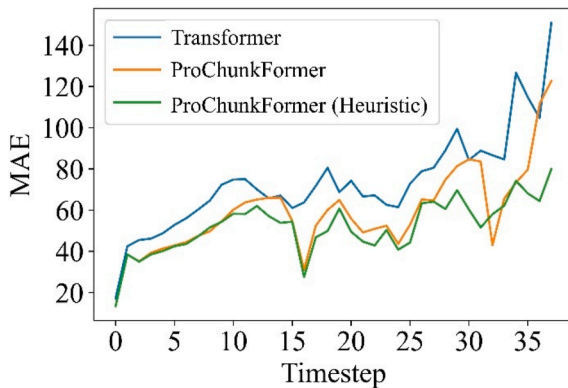


Fig. 14. MAE for the Transformer, ProChunkFormer, and ProChunkFormer (Heuristic) methods evaluated at each time step.

Table 5

Complexity comparison between autoregressive decoding and chunk processing.

Complexity	AD		CP	
	Training	Inference	Training	Inference
Time	$O(T^2)$	$O(T^3)$	$O(T^2/k^2)$	$O(T^3/k^3)$
Space	$O(T^2)$	$O(T^2)$	$O(T^2/k^2)$	$O(T^2/k^2)$

6. Conclusions

In this study, we propose an end-to-end chunked transformer model (ProChunkFormer) for effectively and efficiently reconstructing map-matched, high-quality trajectories from sparse and noisy raw data. The proposed ProChunkFormer leverages chunked processing and parallel computing to increase efficiency while mitigating error

accumulation to improve accuracy—particularly for long-period raw trajectories with long interval times. Additionally, we evaluate the ability of incorporating heuristic prior knowledge, such as the shortest path, to improve performance on long-interval trajectories. While this information does not significantly enhance overall performance, it helps the model converge faster.

CRedit authorship contribution statement

Yonghui Liu: Data curation, Methodology, Formal analysis, Writing – review & editing, Validation, Investigation, Conceptualization, Visualization, Writing – original draft. **Qian Li:** Writing – review & editing, Data curation. **Inhi Kim:** Funding acquisition, Project administration, Supervision, Conceptualization, Writing – review & editing, Resources.

Replication and data sharing

The code and dataset for our proposed model have been uploaded to ETS-Data. For further details, please refer to <https://doi.org/10.26599/ETSD.2025.9190003>.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the National Research Foundation of Korea (NRF), Republic of Korea grant funded by the Korea government (MSIT), Republic of Korea (No. 2021R1C1C1006405).

References

- Abbar, S., Stanojevic, R., Mokbel, M., 2020. STAD: Spatio-temporal adjustment of traffic-oblivious travel-time estimation. In: 2020 21st IEEE International Conference on Mobile Data Management (MDM), pp. 79–88.
- Ali Arman, M., Tampère, C.M.J., 2022. Lane-level trajectory reconstruction based on data-fusion. *Transp. Res. Part C Emerg. Technol.* 145, 103906.
- Bahdanau, D., Cho, K., Bengio, Y., 2016. Neural machine translation by jointly learning to align and translate. <https://doi.org/10.48550/arXiv.1409.0473>.
- Bai, Z., Wu, G., Barth, M.J., Liu, Y., Akin Sisbot, E., Oguchi, K., et al., 2024. A survey and framework of cooperative perception: from heterogeneous singleton to hierarchical cooperation. *IEEE Trans. Intell. Transp. Syst.* 25, 15191–15209.
- Beltagy, I., Peters, M.E., Cohan, A., 2020. Longformer: the long-document transformer. <https://arxiv.org/abs/2004.05150v2>.
- Chen, X., Yin, J., Qin, G., Tang, K., Wang, Y., Sun, J., 2022. Integrated macro-micro modelling for individual vehicle trajectory reconstruction using fixed and mobile sensor data. *Transp. Res. Part C Emerg. Technol.* 145, 103929.
- Chen, Y., Zhang, H., Sun, W., Zheng, B., 2023. RNTrajRec: road network enhanced trajectory recovery with spatial-temporal transformer. In: 2023 IEEE 39th International Conference on Data Engineering (ICDE), pp. 829–842.
- Choi, C., Christensen, H.I., 2013. RGB-D object tracking: a particle filter approach on GPU. In: 2013 IEEE/RSJ International Conference on Intelligent Robots and System, pp. 1084–1091.
- Chondrogianis, T., Bornholdt, J., Bouros, P., Grossniklaus, M., 2022. History oblivious route recovery on road networks. In: Proceedings of the 30th International Conference on Advances in Geographic Information System, pp. 1–10.
- Cui, G., Luo, J., Wang, X., 2018. Personalized travel route recommendation using collaborative filtering based on GPS trajectories. *Int. J. Digit. Earth.* 11, 284–307.
- Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q.V., Salakhutdinov, R., 2019. Transformer-XL: attentive language models beyond a fixed-length context. <https://doi.org/10.48550/arXiv.1901.02860>.
- Devlin, J., Chang, M.W., Lee, K., Toutanova, K., Hulburd, E., Liu, D., et al., 2018. BERT: Pre-training of deep bidirectional transformers for language understanding. <https://arxiv.org/abs/1810.04805v2>.
- Feng, J., Li, Y., Zhang, C., Sun, F., Meng, F., Guo, A., et al., 2018. DeepMove: predicting human mobility with attentional recurrent networks. In: Proceedings of the 2018 World Wide Web Conference, pp. 1459–1468.
- Greenfeld, J.S., 2002. Matching GPS observations to locations on a digital map. In: Transportation Research Board 81st Annual Meeting, pp. 576–582.
- Hoteit, S., Secci, S., Sobolevsky, S., Ratti, C., Pujolle, G., 2014. Estimating human trajectories and hotspots through mobile phone data. *Comput. Netw.* 64, 296–307.
- Krumm, J., Horvitz, E., Letchner, J., 2007. Map matching with travel time constraints. <https://doi.org/10.4271/2007-01-1102>.
- Kurashima, T., Iwata, T., Irie, G., Fujimura, K., 2010. Travel route recommendation using geotags in photo sharing sites. In: Proceedings of the 19th ACM International Conference on Information and Knowledge Management. Toronto ON Canada, pp. 579–588.
- Lan, Z., Liu, L., Fan, B., Lv, Y., Ren, Y., Cui, Z., 2024. Traj-LLM: a new exploration for empowering trajectory prediction with pre-trained large language models. <https://doi.org/10.48550/arXiv.2405.04909>.
- Li, M., Gao, S., Lu, F., Zhang, H., 2019. Reconstruction of human movement trajectories from large-scale low-frequency mobile phone data. *Comput. Environ. Urban Syst.* 77, 101346.
- Li, M., Tong, P., Li, M., Jin, Z., Huang, J., Hua, X.S., 2021. Traffic flow prediction with vehicle trajectories. *Proc. AAAI Conf. Artif. Intell.* 35, 294–302.
- Li, W., Nie, D., Wilkie, D., Lin, M.C., 2017. Citywide estimation of traffic dynamics via sparse GPS traces. *IEEE Intell. Transp. Syst. Mag.* 9, 100–113.
- Liao, H., Shen, H., Li, Z., Wang, C., Li, G., Bie, Y., et al., 2024. GPT-4 enhanced multimodal grounding for autonomous driving: leveraging cross-modal attention with large language models. *Commun. Transp. Res.* 4, 100116.
- Lin, H., Liu, Y., Wang, L., Qu, X., 2025a. A high-precision calibration and evaluation method based on binocular cameras and LiDAR for intelligent vehicles. *IEEE Trans. Veh. Technol.* 1–12.
- Lin, H., Liu, Y., Wang, L., Qu, X., 2025b. Big data-driven advancements and future directions in vehicle perception technologies: from autonomous driving to modular buses. *IEEE Trans. Big Data* 11, 1568–1587.
- Liu, H., Ong, Y.S., Shen, X., Cai, J., 2020. When Gaussian process meets big data: a review of scalable GPs. *IEEE Trans. Neural Netw. Learn. Syst.* 31, 4405–4423.
- Liu, Y., Wu, F., Liu, Z., Wang, K., Wang, F., Qu, X., 2023. Can language models be used for real-world urban-delivery route optimization? *Innovation* 4, 100520.
- Mahmud, D., Hajmohamed, H., Almentheri, S., Alqaydi, S., Aldaheri, L., Khalil, R.A., et al., 2025. Integrating LLMs with ITS: recent advances, potentials, challenges, and future directions. *IEEE Trans. Intell. Transp. Syst.* 26, 5674–5709.
- Mazloumi, E., Currie, G., Rose, G., 2010. Using GPS data to gain insight into public transport travel time variability. *J. Transp. Eng.* 136, 623–631.
- Meng, X., Fu, H., Peng, L., Liu, G., Yu, Y., Wang, Z., et al., 2022. D-LSTM: Short-term road traffic speed prediction model based on GPS positioning data. *IEEE Trans. Intell. Transp. Syst.* 23, 2021–2030.
- Newson, P., Krumm, J., 2009. Hidden Markov map matching through noise and sparseness. In: Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems, pp. 336–343.
- Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., 2018. Improving language understanding by generative pre-training. OpenAI Technical Report.
- Rafati Fard, M., Shariat Mohaymany, A., Shahri, M., 2017. A new methodology for vehicle trajectory reconstruction based on wavelet analysis. *Transp. Res. Part C Emerg. Technol.* 74, 150–167.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., et al., 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.* 21, 1–67.
- Rao, W., Wu, Y.J., Xia, J., Ou, J., Kluger, R., 2018. Origin-destination pattern estimation based on trajectory reconstruction using automatic license plate recognition data. *Transp. Res. Part C Emerg. Technol.* 95, 29–46.
- Ren, H., Ruan, S., Li, Y., Bao, J., Meng, C., Li, R., et al., 2021. MTrajRec: Map-constrained trajectory recovery via Seq2Seq multi-task learning. In: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Minin, pp. 1410–1419.
- Sanaullah, I., Qudus, M., Enoch, M., 2016. Developing travel time estimation methods using sparse GPS data. *J. Intell. Transp. Syst.* 20, 532–544.
- Sang, H., Wang, J., Liu, Q., Chen, W., Zhao, Z., 2024. Physics constrained pedestrian trajectory prediction with probability quantification. *Expert Syst. Appl.* 255, 124743.
- Shi, X., Zhao, D., Yao, H., Li, X., Hale, D.K., Ghiasi, A., 2021. Video-based trajectory extraction with deep learning for High-Granularity Highway Simulation (HIGH-SIM). *Commun. Transp. Res.* 1, 100014.
- Sun, S., Chen, J., Sun, J., 2019. Traffic congestion prediction based on GPS trajectory data. *Int. J. Distrib. Sens. Netw.* 15, 155014771984744.
- Sun, Z., Ban, X.J., 2013. Vehicle trajectory reconstruction for signalized intersections using mobile traffic sensors. *Transp. Res. Part C Emerg. Technol.* 36, 268–283.
- Sutskever, I., Vinyals, O., Le, Q.V., 2014. Sequence to sequence learning with neural networks. <http://arxiv.org/abs/1409.321>.
- Tay, Y., Bahri, D., Yang, L., Metzler, D., Juan, D.C., 2020. Sparse sinkhorn attention. In: Proceedings of the 37th International Conference on Machine Learning, pp. 9438–9447.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., et al., 2017. Attention is all you need. *Adv. Neural Inf. Process. Syst.* 30, 5998–6008.
- Wang, J., Wu, N., Lu, X., Zhao, W.X., Feng, K., 2019. Deep trajectory recovery with fine-grained calibration using Kalman filter. *IEEE Trans. Knowl. Data Eng.* 33, 921–934.
- Wang, Y., Gloudemans, D., Ji, J., Teoh, Z.N., Liu, L., Zachar, G., et al., 2024. Automatic vehicle trajectory data reconstruction at scale. *Transp. Res. Part C Emerg. Technol.* 160, 104520.
- White, C.E., Bernstein, D., Kornhauser, A.L., 2000. Some map matching algorithms for personal navigation assistants. *Transp. Res. Part C Emerg. Technol.* 8, 91–108.
- Woodard, D., Nogin, G., Koch, P., Racz, D., Goldszmidt, M., Horvitz, E., 2017. Predicting travel time reliability using mobile phone GPS data. *Transp. Res. Part C Emerg. Technol.* 75, 30–44.
- Xia, T., Qi, Y., Feng, J., Xu, F., Sun, F., Guo, D., et al., 2021. AttnMove: history enhanced trajectory recovery via attentional network. *Proc. AAAI Conf. Artif. Intell.* 35, 4494–4502.
- Xie, X., van Lint, H., Verbaeck, A., 2018. A generic data assimilation framework for vehicle trajectory reconstruction on signalized urban arterials using particle filters. *Transp. Res. Part C Emerg. Technol.* 92, 364–391.
- Xu, M., Di, Y., Ding, H., Zhu, Z., Chen, X., Yang, H., 2023. AGNP: Network-wide short-term probabilistic traffic speed prediction and imputation. *Commun. Transp. Res.* 3, 100099.
- Yang, S., Wang, B., Shen, Y., Panda, R., Kim, Y., 2023. Gated linear attention transformers with hardware-efficient training. <https://arxiv.org/abs/2312.06635v6>.
- Yuan, J., Zheng, Y., Zhang, C., Xie, X., Sun, G.Z., 2010. An interactive-voting based map matching algorithm. In: 2010 Eleventh International Conference on Mobile Data Management, pp. 43–52.
- Zhang, K., Zhang, H., Dong, Y., Wu, Y., Chen, X., 2023. An ADMM-based parallel algorithm for solving traffic assignment problem with elastic demand. *Commun. Transp. Res.* 3, 100108.
- Zhao, J., Ma, R., Wang, M., 2024. A behaviourally underpinned approach for two-dimensional vehicular trajectory reconstruction with constrained optimal control. *Transp. Res. Part C Emerg. Technol.* 159, 104489.
- Zhao, S., Zhang, K., 2019. A distributionally robust optimization approach to reconstructing missing locations and paths using high-frequency trajectory data. *Transp. Res. Part C Emerg. Technol.* 102, 316–335.
- Zheng, C., Fan, X., Wang, C., Qi, J., 2020. Gman: a graph multi-attention network for traffic prediction. *Proc. AAAI Conf. Artif. Intell.* 34, 1234–1241.
- Zheng, Y., 2015. Trajectory data mining. *ACM Trans. Intell. Syst. Technol.* 6, 1–41.



Yonghui Liu received the B.S. degree in oceanography engineering from Zhejiang University in 2021. He is currently pursuing the Ph.D. degree in transportation at the Cho Chun Shik Graduate School of Mobility, Korea Advanced Institute of Science and Technology, Republic of Korea. His research interests include deep learning, spatiotemporal modeling, and urban computing.



Qian Li received the B.S. degree in oceanography engineering from Zhejiang University in 2021. He is currently working as an engineer at Donghai Laboratory. His research interests include deep learning, digital twins, and simulation.



Inhi Kim received the Ph.D. degree from the Department of Civil Engineering, The University of Queensland, Australia, in 2014. He is currently an Associate Professor at the Cho Chun Shik Graduate School of Mobility, Korea Advanced Institute of Science and Technology, Republic of Korea. He has particular interests and skills in the areas of deep learning, visualization, demand forecasting, strategic planning, traffic signal operation, public transportation planning, safety in conjunction with the integration of traffic simulation and driving simulators, and the evaluation of connected cars in terms of safety and efficiency.